

FULL STACK AI ENGINEER INTERN PROJECT SUBMISSION

AI Prompts Used During Development

A curated, transparent record of materially significant engineering prompts used with AI development tools across the full-stack lifecycle — distinguishing authentic developer prompts from consolidated engineering tasks.

DEVELOPER

Sathwik Bodakunta

PROJECT

AiProf Study Companion

PRODUCTION URL

ai-prof-study-companion.vercel.app

REPOSITORY

github.com/sathwik1821/Ai-Prof-Study-Companion

Objective & Dual-Form Prompt Documentation Standards

This document presents a **curated, professionally contextualized record of the most materially significant prompts used during the development** of the AiProf Study Companion, drawn directly from recorded Antigravity IDE development transcripts. Rather than presenting an unedited transcript of casual short messages or fabricating artificially verbose prompts, this document uses two clearly defined standards:

1. Historical Prompts — Actually Used

Preserves the original wording and informal developer phrasing from the session logs without artificial embellishment.

2. Consolidated Engineering Prompts

Where multiple brief messages formed a cohesive engineering task, the combined intent is accurately formulated and explicitly labeled *not verbatim*.

Security & Redaction: All production credentials, database connection secrets, API tokens, and personal email addresses have been redacted using [REDACTED] or [REDACTED EMAIL] .

Development Domain Coverage Matrix

DOMAIN	PROMPTS	KEY ENGINEERING SCOPE	TECHNOLOGIES IMPACTED
A. Architecture & System Design	Prompts 01–02	3-tier cloud topology & evaluation rubric feature audit	Vercel, Render, Neon, Spring Boot, React 19
B. Frontend Engineering	Prompts 03–05	Space/Project metadata editing, sidebar UX, Targeted Mastery cards	React 19, Lucide Icons, Vanilla CSS Grid
C. Backend Engineering	Prompts 06–07	Role-based access control (RBAC: USER vs ADMIN), admin telemetry guards	Spring Security 6.4, JWT Filter, Spring Data JPA
D. Database & Data Layer	Prompts 08–09	Neon PostgreSQL 16 setup, pgvector 3072d index, cascade deletion SQL	Neon Serverless, pgvector, Flyway, Hibernate
E. AI / LLM Engineering	Prompts 10–11	Dynamic concept mastery algorithm, Empty Materials RAG guard gate	Gemini Flash Lite, Gemini Embeddings, pgvector
F. Authentication & Security	Prompts 12–13	Google OAuth 2.0 Identity Services, 6-digit email OTP verification gate	Google ID Token Verifier, Resend API, BCrypt
G. Debugging & Problem Solving	Prompts 14–16	OAuth Error 400 origin mismatch, SMTP port 587 block, React Error #310	Google Cloud Console, Resend REST, React Hooks
H. Testing & Validation	Prompts 17–18	End-to-end vector pipeline validation, live production DOM browser audit	Chrome Headless, JUnit 5, Browser Subagent
I. Deployment & Configuration	Prompts 19–20	Production OAuth origin sync, Vercel Deployment Protection bypass	Vercel CLI, Render Docker, GitHub CI/CD
J. Documentation	Prompts 21–23	Public submission README, Architecture PDF, AI Tools & Usage PDF	Markdown, Headless Chrome Engine, Node.js

Page 1 of 10

A. Architecture & System Design

ARCHITECTURE

Prompt 01 — Full 3-Tier Cloud Deployment Architecture Strategy CONSOLIDATED FROM DEVELOPMENT CONVERSATION

"explain whole deployment step by step iam a beggineer lets move as i say i completed this"

Consolidated Engineering Prompt — not verbatim: "Architect and establish a production cloud deployment strategy for the AiProf full-stack platform. Decompose the system into a decoupled three-tier topology: a React SPA hosted on Vercel edge networks, a containerized Spring Boot backend running in Docker on Render, a serverless PostgreSQL database with pgvector on Neon, and external Gemini AI services. Provide an incremental rollout plan."

ENGINEERING OBJECTIVE:	Establish an enterprise-grade decoupled cloud topology separating static asset delivery, backend compute, vector storage, and LLM inference.
AI CONTRIBUTION:	Drafted multi-stage Dockerfile for Spring Boot, configured cross-origin policies, formulated deployment sequences, and mapped environment variables across Vercel, Render, and Neon.
DEVELOPER CONTRIBUTION:	Evaluated cloud hosting tiers, provisioned Vercel, Render, and Neon workspaces, linked GitHub repositories, and validated container spin-up.

Prompt 02 — Comprehensive Architectural & Feature Audit Against Rubric ACTUAL PROMPT

"according to this what features we covered and what not and if all covered we will move on to the next thing documentation and demo creation i will say so so dont start anything now itself(documentation)"

ENGINEERING OBJECTIVE:	Conduct a formal architectural gap analysis comparing implemented application features against the internship evaluation criteria before freezing the codebase.
AI CONTRIBUTION:	Audited repository modules against rubric requirements (RAG accuracy, Socratic tutoring, token efficiency, RBAC, telemetry, and rate-limiting).
DEVELOPER CONTRIBUTION:	Prioritized Google OAuth 2.0 and email OTP security hardening over minor UI cosmetics and explicitly enforced a code-freeze guardrail during documentation.

B. Frontend Engineering

FRONTEND

Prompt 03 — Space & Project Metadata Editing UI

ACTUAL PROMPT

"also add the ability to change the space project name descriptions"

ENGINEERING OBJECTIVE:	Enable students to dynamically edit and update names and descriptions for workspaces and study projects with instant optimistic UI feedback.
AI CONTRIBUTION:	Implemented modal dialog components in <code>WorkspaceView.jsx</code> and <code>ProjectView.jsx</code> , wired PUT API mutation handlers, and handled form validation.
DEVELOPER CONTRIBUTION:	Tested inline name updating across multiple test spaces, validated form input boundaries, and confirmed state synchronization across sidebar navigation.

Prompt 04 — Streamlining Navigation & Account Settings in Sidebar UX

ACTUAL PROMPT

"i think there is no need of adding section for profile in the side bar as we can access it right from the below where name is shown"

ENGINEERING OBJECTIVE:	Streamline sidebar navigation ergonomics by eliminating redundant menu items and consolidating user account settings into the bottom identity drawer.
AI CONTRIBUTION:	Refactored <code>Sidebar.jsx</code> , removed top-level profile links, and added an interactive popover menu to the user avatar drawer with logout triggers.
DEVELOPER CONTRIBUTION:	Evaluated visual layout on laptop and mobile screen widths, approved the consolidated navigation hierarchy, and confirmed reduced visual clutter.

Prompt 05 — Dynamic Dashboard Reorganization for Targeted Concept Mastery

ACTUAL PROMPT

"remove setup guide completely and add the recommended thing(targeted mastery u added in the analytics) right next to the continue in the dashboard"

ENGINEERING OBJECTIVE:	Transform the student dashboard from static onboarding checklists into an action-oriented study hub featuring dynamic AI-driven Targeted Mastery recommendations.
AI CONTRIBUTION:	Redesigned <code>DashboardPage.jsx</code> CSS grid layout, removing static cards and placing the Targeted Mastery card directly beside "Continue Studying".
DEVELOPER CONTRIBUTION:	Verified dynamic card rendering when concept mastery scores updated after quiz submissions and validated responsive breakpoints on desktop and mobile.

C. Backend Engineering

BACKEND

Prompt 06 — Enforcing Strict Role-Based Access Control (RBAC)

ACTUAL PROMPT

"lets perform what u have added one after another and the first thing to do is now there is no seperation between normal user and the admin ie everyone is an admin correct it first"

ENGINEERING OBJECTIVE:	Eliminate privilege escalation vulnerabilities by establishing strict role boundaries between regular students (<code>ROLE_USER</code>) and system administrators (<code>ROLE_ADMIN</code>).
AI CONTRIBUTION:	Updated Spring Security filter chain configurations, refactored <code>User</code> entity role initialization to assign <code>ROLE_USER</code> by default, and secured API routes using method-level security.
DEVELOPER CONTRIBUTION:	Created separate test accounts (<code>ROLE_USER</code> vs <code>ROLE_ADMIN</code>), verified that non-admin tokens received HTTP 403 Forbidden on administrative endpoints, and confirmed database role persistence.

Prompt 07 — Securing Administrative Telemetry & API Endpoints

ACTUAL PROMPT

"the thing in this is other account has admin tab but all the details are set to 0"

ENGINEERING OBJECTIVE:	Prevent information leakage and unauthorized UI exposure by enforcing backend <code>@PreAuthorize</code> guards on telemetry APIs and hiding admin navigation elements from non-admin principals.
AI CONTRIBUTION:	Added <code>@PreAuthorize("hasRole('ADMIN')")</code> annotations across <code>AdminController.java</code> , updated JWT claims parsing to extract authorities, and conditionally rendered the Admin UI tab in React.
DEVELOPER CONTRIBUTION:	Tested cross-account authentication states, verified that standard student accounts no longer see administrative tabs, and validated zero telemetry exposure in network inspector logs.

D. Database & Data Layer

DATABASE

Prompt 08 — Cloud PostgreSQL Provisioning & pgvector Extension Setup

CONSOLIDATED FROM DEVELOPMENT CONVERSATION

"before pg vector we have to do for our main db right"
[Followed by Neon CLI project setup instructions: `neon link --project-id dry-mouse-00052621 --branch production`, connection string configuration, and `pgvector` extension verification]

Consolidated Engineering Prompt — not verbatim: "Provision and configure a serverless PostgreSQL 16 database on Neon for the production backend. Configure pooled connection URLs with mandatory SSL encryption, write Flyway migration scripts to initialize the relational schema, and activate the `pgvector` extension to store 3072-dimensional document embeddings."

ENGINEERING OBJECTIVE:	Provision managed relational and vector persistence with automated schema version control and connection pooling.
AI CONTRIBUTION:	Authored Flyway migration <code>V1__init.sql</code> with <code>CREATE EXTENSION IF NOT EXISTS vector;</code> , configured <code>HikariCP</code> connection pooling with <code>sslmode=require</code> in <code>application.yml</code> , and defined JPA entity mappings.
DEVELOPER CONTRIBUTION:	Executed Neon CLI linkage, configured Neon connection secrets in Render dashboard, monitored Flyway migration execution during boot, and inspected schema tables in Neon console.

Prompt 09 — Foreign Key Cascade Integrity & Data Cleanup Queries

ACTUAL PROMPT

"give the neon db query to delete the user with the email [REDACTED EMAIL]"

ENGINEERING OBJECTIVE:	Safely delete test user records from the production database while verifying referential integrity and cascade deletion behavior across spaces, projects, documents, chunks, and chat sessions.
AI CONTRIBUTION:	Formulated safe, transactional SQL deletion scripts ordering child entity cleanup (<code>user_roles</code> , <code>document_chunks</code> , <code>study_materials</code> , <code>chat_sessions</code> , <code>projects</code> , <code>spaces</code> , <code>users</code>) respecting foreign key constraints.
DEVELOPER CONTRIBUTION:	Executed queries in Neon SQL console, verified that deleting the user cleanly removed all associated vector embeddings and materials without leaving orphaned rows, and re-tested registration.

E. AI / LLM Engineering

AI / LLM

Prompt 10 — Dynamic Cognitive Mastery Scoring Algorithm

ACTUAL PROMPT

"another issue is when i create a new space and project it autotmatically has 35 percentage and the whole project is diving into 3 Spring AI Fundamentals Core definitions, terminology, and key foundational principles... what could be the better way to handle this"

ENGINEERING OBJECTIVE:	Replace arbitrary mock mastery percentages with a dynamic, data-driven concept mastery calculation reflecting genuine student quiz accuracy and study interaction depth.
AI CONTRIBUTION:	Designed and coded an adaptive calculation algorithm in <code>AnalyticsService.java</code> and <code>WorkspaceService.java</code> computing mastery scores (0%–100%) based on real quiz attempt scores, evaluation feedback, and study frequency.
DEVELOPER CONTRIBUTION:	Validated the updated scoring by completing quizzes with known scores (100% vs 50%), verifying that mastery percentages accurately reflected assessment performance without hardcoded defaults.

Prompt 11 — Grounded RAG Precondition Gate (Empty Materials Prevention)

ACTUAL PROMPT

"quiz and tutor ask to upload doc if there is no single document uploaded in the project add the my spaces on to the side bar remove it from the dashbaord"

ENGINEERING OBJECTIVE:	Enforce strict grounding prerequisites preventing LLM hallucinations by blocking AI Tutor and Quiz generation until at least one syllabus or study document has been uploaded to the project.
AI CONTRIBUTION:	Implemented client-side and server-side validation gates (<code>materials.isEmpty()</code>) returning descriptive empty-state warnings and guiding students to upload course materials before invoking Gemini.
DEVELOPER CONTRIBUTION:	Tested empty projects and populated projects, verified that ungrounded prompt execution was blocked, and confirmed that grounding citations rendered accurately once documents were uploaded.

F. Authentication & Security

SECURITY

Prompt 12 — Google OAuth 2.0 Identity Services Integration

ACTUAL PROMPT

"they have metoined security is main proroity so lets keep google auth"

ENGINEERING OBJECTIVE:	Implement enterprise-grade Google OAuth 2.0 authentication to streamline student onboarding while ensuring cryptographic token validation on the backend.
AI CONTRIBUTION:	Integrated <code>@react-oauth/google</code> on the client, implemented <code>GoogleAuthService.java</code> using Google API Client libraries for ID token signature and audience verification, and handled account provisioning.
DEVELOPER CONTRIBUTION:	Configured Google Cloud Console OAuth credentials, registered authorized JavaScript origins and redirect URIs, configured client ID in Vercel environment variables, and validated end-to-end Google sign-in.

Prompt 13 — 6-Digit Email OTP Verification Security Gate

ACTUAL PROMPT

"also add the otp veriification on singup otp should come to mail"

ENGINEERING OBJECTIVE:	Secure user registration against bot spam and unverified accounts by requiring a timed 6-digit one-time password delivered to the student's email before account activation.
AI CONTRIBUTION:	Built <code>OtpService.java</code> with cryptographically secure 6-digit generation, 5-minute expiration, in-memory caching, rate limiting, and an authentication state machine gating account activation.
DEVELOPER CONTRIBUTION:	Tested registration flow with real email accounts, verified expired OTP rejection, confirmed rate-limiting behavior, and approved the multi-step verification UX.

G. Debugging & Problem Solving

DEBUGGING

Prompt 14 — Google OAuth 2.0 Error 400: origin_mismatch Resolution

ACTUAL PROMPT

"You can't sign in to this app because it doesn't comply with Google's OAuth 2.0 policy.If you're the app developer, register the JavaScript origin in the Google Cloud Console.[Learn more about this error] (<https://developers.google.com/identity/protocols/oauth2/javascript-implicit-flow#authorization-errors-origin-mismatch>)
If you are a developer of this app, see error details.
Error 400: origin_mismatch"

ENGINEERING OBJECTIVE: Diagnose and resolve a Google OAuth 2.0 policy rejection occurring on newly deployed Vercel domain URLs.

AI CONTRIBUTION: Diagnosed that the active Vercel preview domain was not registered in Google Cloud Console's "Authorized JavaScript origins"; provided step-by-step resolution to add both canonical and preview domains.

DEVELOPER CONTRIBUTION: Accessed Google Cloud Console, updated authorized web client origins with `https://ai-prof-study-companion.vercel.app`, and confirmed successful Google login in production.

Prompt 15 — Cloud SMTP Egress Block Diagnosis & Resend REST API Migration

CONSOLIDATED FROM DEVELOPMENT CONVERSATION

"Check Your Inbox
We sent a 6-digit security code to [REDACTED EMAIL] i didnt recived it yet
...
cant we make it work"

Consolidated Engineering Prompt — not verbatim: "Investigate why email OTP verification codes are not being delivered in the cloud production environment despite working locally. Diagnose network or cloud provider firewall restrictions on SMTP egress ports, and implement a resilient HTTPS REST API email delivery transport that bypasses cloud port blocks."

ENGINEERING OBJECTIVE: Resolve production email delivery failure caused by cloud hosting firewall restrictions on outbound SMTP ports (25, 465, 587).

AI CONTRIBUTION: Identified that Render blocks outbound SMTP by default; engineered `ResendEmailService.java` using Resend's HTTPS REST API over port 443 with automatic fallback to standard SMTP.

DEVELOPER CONTRIBUTION: Created a Resend account, provisioned an API key, configured `RESEND_API_KEY` on Render, and verified instantaneous OTP delivery to student inboxes.

Prompt 16 — Resolving Minified React Error #310 (Conditional Hook Invocation)

CONSOLIDATED FROM DEVELOPMENT CONVERSATION

[Screenshot & Runtime Console Trace]
"Minified React Error #310: Rendered more hooks than during the previous render.
at Object.Qs [as useMemo] ... at DashboardPage"

Consolidated Engineering Prompt — not verbatim: "Debug a runtime crash on the production dashboard triggering Minified React Error #310. Analyze `DashboardPage.jsx` to identify any violation of the Rules of Hooks, such as early return statements executing before `useMemo` or `useEffect` declarations, and refactor the component to ensure unconditional hook execution."

ENGINEERING OBJECTIVE: Fix a critical runtime blank-screen bug on `DashboardPage.jsx` caused by conditional hook execution order.

AI CONTRIBUTION: Traced the error to an early `if (loading) return <Spinner />` placed before `useMemo` declarations; refactored hook declarations to the top level before conditional returns.

DEVELOPER CONTRIBUTION: Rebuilt frontend bundle (`npm run build`), tested dashboard rendering across authenticated and unauthenticated states, and verified zero console warnings.

H. Testing & Validation

TESTING & VALIDATION

Prompt 17 — Functional Validation of End-to-End AI Pipelines

ACTUAL PROMPT

"i ensured that ai features are working well"

ENGINEERING OBJECTIVE:	Execute comprehensive end-to-end verification of document ingestion, vector embedding generation, pgvector retrieval, Socratic tutor responses, and quiz generation.
AI CONTRIBUTION:	Formulated test validation scenarios covering semantic chunking, embedding generation via <code>gemini-embedding-001</code> , vector distance thresholding, and JSON schema compliance for quizzes.
DEVELOPER CONTRIBUTION:	Uploaded sample academic course notes, initiated interactive tutor sessions, verified citation source accuracy, took AI-generated quizzes, and validated automated grading.

Prompt 18 — Live Autonomous Browser Deployment Validation

ACTUAL PROMPT

"https://ai-prof-study-companion-git-main-sathwik-5cbb.vercel.app/dashboard"

ENGINEERING OBJECTIVE:	Validate live production environment stability, authentication session restoration, and responsive UI layout via automated browser DOM inspection.
AI CONTRIBUTION:	Deployed an autonomous browser subagent to navigate to the production deployment URL, inspect DOM element hierarchies, evaluate JavaScript console error logs, and capture visual status screenshots.
DEVELOPER CONTRIBUTION:	Reviewed browser session recordings and screenshots, confirmed correct token restoration from <code>localStorage</code> , and approved production release readiness.

I. Deployment & Configuration

DEPLOYMENT & CONFIG

Prompt 19 — Production OAuth Origin Re-Alignment Across Domain Aliases

ACTUAL PROMPT

"the sign in with google also not working after updating the vercel link what should i do"

ENGINEERING OBJECTIVE:	Synchronize Google OAuth credentials, Vercel frontend environment variables, and Spring Boot backend CORS configurations after assigning a production domain alias.
AI CONTRIBUTION:	Provided an exact audit of required configuration updates across <code>VITE_GOOGLE_CLIENT_ID</code> , Google Cloud Console Authorized Origins, and Spring Security <code>allowedOrigins</code> patterns.
DEVELOPER CONTRIBUTION:	Updated client environment settings, verified backend CORS filters accepted requests from the new domain alias, and tested Google Sign-In across browsers.

Prompt 20 — Disabling Vercel Deployment Protection for Public Access

ACTUAL PROMPT

"in vercel app if opened by some one else it is asking them to req for access how to solve this"

ENGINEERING OBJECTIVE:	Eliminate Vercel's automated authentication barrier (SSO / Deployment Protection) to ensure evaluators can freely access and interact with the live application without a team invite.
AI CONTRIBUTION:	Identified that Vercel's "Deployment Protection" was intercepting requests with an access request modal; provided exact settings navigation path (<code>Settings -> Deployment Protection -> Vercel Authentication</code>) to disable it.
DEVELOPER CONTRIBUTION:	Configured Vercel project settings to disable deployment protection, verified public accessibility in private/incognito browser windows, and confirmed external usability.

J. Documentation

DOCUMENTATION

Prompt 21 — Repository Submission Preparation, README & Secret Sanitization

ACTUAL PROMPT

"so i have to submit this project repo so better i would have neat readme and things Public
Contains the complete project
Has a proper README.md
Has clean project structure
Doesn't contain secrets (.env, passwords, API keys, etc.)
Includes backend + frontend + relevant configuration
Can be cloned and run by the evaluator
Has meaningful commit history if possible DONT CHANGE THE Application as it is perfect now"

ENGINEERING OBJECTIVE:	Prepare the entire repository for official evaluator review by structuring a comprehensive README, verifying local clone-and-run workflows, sanitizing secrets, and safeguarding working application code.
AI CONTRIBUTION:	Crafted production-grade README.md , replaced hardcoded credentials with .env.example templates, sanitized application.yml , verified local build scripts, and drafted setup documentation.
DEVELOPER CONTRIBUTION:	Reviewed all sanitized configuration files, tested local build commands (npm run build , ./mvnw test-compile), committed the repository, and pushed the clean public codebase to GitHub.

Prompt 22 — System Architecture Specification Document Generation CONSOLIDATED FROM DEVELOPMENT CONVERSATION

"Create the Architecture Documentation PDF required for my Full Stack AI Engineer Intern project submission. You already have the complete context of the project because you designed and implemented it. Use the actual technologies, architecture, database schema, APIs, authentication flow, AI integration, frontend/backend structure, and deployment setup from our project. Do NOT invent anything..."

Consolidated Engineering Prompt — not verbatim: "Generate a publication-grade System Architecture Documentation PDF for the AiProf platform. Document the decoupled three-tier topology, request-response lifecycles, complete ASCII flowcharts, entity-relationship database schemas, Spring Security filter chains, pgvector similarity query mechanics, and production cloud infrastructure."

ENGINEERING OBJECTIVE:	Author an exhaustive technical reference document detailing system topology, data models, and architectural decisions.
AI CONTRIBUTION:	Structured, authored, and compiled AiProf_Architecture_Documentation.pdf utilizing Chrome headless printing with professional diagrams and tables.
DEVELOPER CONTRIBUTION:	Reviewed technical accuracy, verified database schema representations and API endpoint mappings, and approved the document for intern submission.

Prompt 23 — Dual-Category AI Tools & Usage Documentation Generation

CONSOLIDATED FROM DEVELOPMENT CONVERSATION

"Create the AI Tools & Usage Documentation PDF required for my Full Stack AI Engineer Intern project submission.
The document must be transparent and factually accurate. Clearly separate two types of AI usage: AI USED TO BUILD THE PRODUCT (tools that assisted me as the developer) vs AI FEATURES WITHIN THE PRODUCT (functionality built into the final application)..."

Consolidated Engineering Prompt — not verbatim: "Generate a transparent AI Tools & Usage Documentation PDF explicitly separating AI tools utilized by the developer during the engineering lifecycle (Antigravity IDE, agent subagents, Chrome browser tools) from AI models and pipelines embedded inside the student-facing product (Gemini Flash Lite, gemini-embedding-001, pgvector). Document prompt engineering techniques, cost/token efficiency, and human oversight guardrails."

ENGINEERING OBJECTIVE:	Provide a transparent, audited technical report detailing developer AI assistance contrasted with the production application's AI capabilities.
AI CONTRIBUTION:	Authored and compiled AiProf_AI_Tools_and_Usage_Documentation.pdf , establishing clear categorization between development assistance and application features.
DEVELOPER CONTRIBUTION:	Verified transparency reporting, confirmed accurate model specifications, and approved the document for intern submission.

Engineering Prompt Patterns Demonstrated

METHODOLOGY

Analysis of Development Methodologies Across Full-Stack AI Engineering

Reviewing the chronological prompts across the AiProf development lifecycle reveals ten distinct engineering prompt patterns. Rather than using AI as a basic chatbot or unguided code generator, the developer employed AI as an interactive, pair-programming engineering collaborator bounded by technical constraints, live verification, and architectural discipline.

1. Requirements → Implementation

Translating business and internship rubric requirements directly into working full-stack features (e.g., Socratic tutoring, concept mastery calculation, interactive quiz generation) without unnecessary intermediate boilerplate.

2. Architecture → Implementation

Structuring decoupled multi-tier systems (React 19 SPA + Spring Boot 3 + Neon PostgreSQL pgvector) and guiding the incremental step-by-step rollout across cloud infrastructure.

3. Feature Refinement & UX Iteration

Iteratively optimizing interface ergonomics based on real user testing, such as relocating profile settings into the user avatar drawer and elevating Targeted Mastery cards on the student dashboard.

4. Constraint-Driven Development

Enforcing strict operational boundaries on the AI assistant, prominently demonstrated by commands like `"DONT CHANGE THE Application as it is perfect now"` to prevent accidental regression during documentation.

5. Error-Driven Debugging & Root Cause Isolation

Providing exact runtime stack traces (`Minified React Error #310` , `Error 400: origin_mismatch`) and symptoms (`"otp is not coming"`), enabling immediate root-cause isolation and minimal diff patches.

6. Security Hardening & Zero-Trust Verification

Actively auditing role boundaries (`"everyone is an admin correct it first"`), guarding telemetry endpoints with `@PreAuthorize` , and implementing two-factor email OTP and Google OAuth 2.0 verification.

7. AI/RAG Pipeline Design & Hallucination Defense

Enforcing strict prerequisite gates (blocking tutor and quiz requests when document count is zero) and calibrating cosine similarity thresholds to guarantee grounded, cited LLM responses.

8. Autonomous Testing & Live Browser Validation

Deploying autonomous browser subagents and terminal compilation tests to inspect live production URLs, verify local storage JWT restoration, and confirm zero console warnings.

9. Production Deployment & Cloud Sync

Managing multi-cloud environment variables, coordinating OAuth authorized origins across preview and production URLs, and bypassing platform-level SSO deployment gates for public access.

10. Submission Packaging & Secret Sanitization

Preparing clean, evaluator-ready public repositories with thorough documentation, reproducible local setup instructions, and zero committed production credentials.

Representative Engineering Prompt Patterns (Reference Only)

EXAMPLE PATTERNS

Notice: Reference Architecture Prompts — Not Historical Transcripts
The following prompt templates illustrate mature engineering prompt formulations recommended for enterprise full-stack AI development. They are provided as reference benchmarks and were not historical prompts used in this project session.

Representative Architecture Review Prompt (Example, not historical)
"Review the current three-tier architecture (React SPA + Spring Boot + Neon PostgreSQL pgvector) and identify coupling, security, scalability, and deployment risks. Propose concrete refactoring steps that preserve our zero-breaking-change rule, minimize external cloud costs, and satisfy the internship project specifications."

Representative RAG Optimization Prompt (Example, not historical)
"Analyze the document chunking and vector retrieval pipeline in DocumentChunkService.java. Compare recursive text splitting with semantic heading-aware chunking for academic textbook ingestion. Implement pgvector HNSW indexing on vector(3072) and benchmark cosine distance query latency under simulated concurrent student query loads."

Submission Verification & Deliverables Summary

DELIVERABLES

Submission Document	Filename	Format & Engine	Status & Verification
Artifact I: System Architecture	AiProf_Architecture_Documentation.pdf	PDF (Headless Chrome)	Completed • Published to GitHub repository root
Artifact II: AI Tools & Usage	AiProf_AI_Tools_and_Usage_Documentation.pdf	PDF (Headless Chrome)	Completed • Published to GitHub repository root
Artifact III: Prompts Used	AiProf_AI_Prompts_Used_During_Development.pdf	PDF (Headless Chrome)	Completed • 23 Prompts across 10 Domains + Synthesis
Production Application	ai-prof-study-companion.vercel.app	Live Web Platform	Publicly accessible • Zero SSO gate • Fully verified

AiProf Study Companion • AI Prompts Used During Development • Prepared for Full Stack AI Engineer Intern Project Submission
All historical prompts faithfully preserved from authentic Antigravity IDE development session logs • Confirmed and validated by Sathwik Bodakunta